TypeScript Tutorial: Adding Type Safety to JavaScript

Introduction

- As we learned from Sarah's story, TypeScript saved her from countless runtime errors and debugging sessions.
- This tutorial will teach you how TypeScript adds type safety to JavaScript while maintaining all of JavaScript's power and flexibility.

What is TypeScript?

- TypeScript is a **superset of JavaScript** that adds **static type checking**.
- Remember Sarah's 3 AM debugging session? TypeScript would have caught many of those errors at compile time!

Why TypeScript?

```
// JavaScript - Error found at runtime (in production!)
function calculatePriority(task) {
    // What if priority is undefined?
    return task.priority.level + task.priority.urgency;
}

// TypeScript - Error found during development
interface Task {
    // Optional property
    priority?: { level: number; urgency: number };
}

function calculatePriority(task: Task): number {
    // TypeScript ERROR: 'task.priority' is possibly 'undefined'.
    return task.priority.level + task.priority.urgency;
}
```

Setting Up TypeScript

We need to install Typescript compiler (tsc)!

```
# Install the latest TypeScript version globally
npm install -g typescript@latest

# Create a TypeScript file
echo 'console.log("Hello, TypeScript!");' > hello.ts

# Compile to JavaScript
tsc hello.ts

# Run the compiled JavaScript
node hello.js
```

```
❯  tsc --version
Version 7.0.2
```

Make sure you use TypeScript 5.0+.

We need `tsconfig.json` to configure typescript compiler:

```
{
  "compilerOptions": {
    "target": "ES2018",
    "module": "es2015",
    "lib": [ "ES2018", "DOM" ],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": [ "src/**/*" ],
  "exclude": [ "node_modules" ]
}
```

Basic Syntax and Type Annotations

Variables with Types

```
// Type annotations - explicitly declare types
let studentName: string = "Alice";
let age: number = 20;
let isEnrolled: boolean = true;

// Type inference - TypeScript figures out the type
let courseName = "CSC640";  // TypeScript knows this is string
let creditHours = 3;         // TypeScript knows this is number

// Constants with types
const PASSING_GRADE: number = 60;
const UNIVERSITY_NAME: string = "Tech University";
```

Type Errors at Compile Time

```
let score: number = 85;

// This won't compile!
score = "Eighty-five";
// Error: Type 'string' is not assignable to type 'number'

// This helps catch bugs early
let studentCount: number = 25;
// studentCount = studentCount + "1";
// Error! Would be "251" in JavaScript
studentCount = studentCount + 1;      // Correct: 26
```

Types Disappear After Compilation

TypeScript:

```
interface Point { x: number; y: number }

function distance(p: Point): number {
    return Math.sqrt(p.x ** 2 + p.y ** 2);
}
```

Compiled JavaScript (output of `tsc`):

```
function distance(p) {
    return Math.sqrt(p.x ** 2 + p.y ** 2);
}
```

- `interface Point` and every type annotation are **removed**.
- Types exist only for the compiler. Nothing checks them at runtime, so data from outside (user input, network) still needs runtime checks.

Union Types

```
// Variable can be multiple types
let result: number | string;
result = 100;         // OK
result = "Perfect";   // OK
// result = true;     // Error!

// Useful for parameters that accept more than one kind of value
function formatId(id: number | string): string {
    if (typeof id === "number") {
        return `ID-${id}`;    // id is a number here
    }
    return id.toUpperCase();  // id is a string here
}

formatId(12345);  // "ID-12345"
formatId("abc");  // "ABC"
```

Type System

As its name suggests, TypeScript is built around a type system.

Basic Types

```
// Primitives
let studentName: string = "Sarah";
let age: number = 22;
let isActive: boolean = true;
let nickname: string | null = null;            // null must be allowed explicitly
let middleName: string | undefined = undefined;

// Arrays
let scores: number[] = [85, 92, 78];
let names: string[] = ["Alice", "Bob", "Carol"];
let mixed: Array<string | number> = ["Alice", 20, "Bob", 21];
```

```
// Tuple - fixed length array with known types
let student: [string, number] = ["Alice", 20];
// student = [20, "Alice"];  // Error! Wrong order

// Enum - named constants
enum GradeCutoff {
    A = 90,
    B = 80,
    C = 70,
    D = 60,
    F = 0
}

let myCutoff: GradeCutoff = GradeCutoff.B;

// Tip: a union of string literals (next slide) is often simpler than an enum
```

Type Aliases

```
// Create custom type names
type StudentID = number;
type GPA = number;
type CourseName = string;

// Use them for clarity
let id: StudentID = 12345;
let gpa: GPA = 3.85;
let course: CourseName = "TypeScript 101";

// Complex type alias
type LetterGrade = "A" | "B" | "C" | "D" | "F";
let finalGrade: LetterGrade = "A";
// finalGrade = "E";  // Error! "E" is not a valid grade
```

Interfaces

```
// Define object structure
interface Student {
    id: number;
    name: string;
    email: string;
    gpa: number;
    courses: string[];
}

// Use the interface
const alice: Student = {
    id: 1,
    name: "Alice Johnson",
    email: "alice@university.edu",
    gpa: 3.8,
    courses: ["CS101", "MATH201"]
};

// TypeScript ensures all properties are present
// const bob: Student = {
//     id: 2,
//     name: "Bob Smith"
//     // Error! Missing email, gpa, courses
// };
```

Optional and Readonly Properties

```
interface Course {
    code: string;
    name: string;
    credits: number;
    instructor?: string;       // Optional property
    readonly id: number;       // Can't be changed after creation
}

const seCourse: Course = {
    id: 101,
    code: "ASE285",
    name: "Software Engineering",
    credits: 3
    // instructor is optional, so this is valid
};

// seCourse.id = 102;
// Error! Cannot assign to 'id' because it is a read-only property
seCourse.instructor = "Dr. Smith";
// OK - adding optional property
```

Functions with Types

TypeScript lets you add types to both traditional and arrow functions.

Function Type Annotations

Parameters and return types

```
function calculateGPA(grades: number[], credits: number[]): number {
    let totalPoints = 0;
    let totalCredits = 0;

    for (let i = 0; i < grades.length; i++) {
        totalPoints += grades[i] * credits[i];
        totalCredits += credits[i];
    }

    return totalPoints / totalCredits;
}
```

Arrow function with types

```
// Arrow function with types
const getLetterGrade = (percentage: number): string => {
    if (percentage >= 90) return "A";
    if (percentage >= 80) return "B";
    if (percentage >= 70) return "C";
    if (percentage >= 60) return "D";
    return "F";
};
```

Optional and Default Parameters

Optional parameter (?)

```
function greetStudent(name: string, title?: string): string {
    if (title) {
        return `Hello, ${title} ${name}!`;
    }
    return `Hello, ${name}!`;
}

console.log(greetStudent("Alice"));          // "Hello, Alice!"
console.log(greetStudent("Bob", "Dr."));     // "Hello, Dr. Bob!"
```

Default parameters (=)

```
function createCourse(
    code: string,
    name: string,
    credits: number = 3,
    instructor: string = "TBA"
): Course {
    return { id: Date.now(), code, name, credits, instructor };
}

createCourse("ASE285", "Software Engineering");
// credits is 3 and instructor is "TBA"
createCourse("CSC494", "IoT Systems", 4, "Dr. Cho");
```

Function Types

```
// Define a function type
type MathOperation = (a: number, b: number) => number;

// Use the function type
const add: MathOperation = (a, b) => a + b;
const multiply: MathOperation = (a, b) => a * b;

// Function that takes a function as parameter
function calculate(
    a: number,
    b: number,
    operation: MathOperation
): number {
    return operation(a, b);
}

console.log(calculate(5, 3, add));        // 8
console.log(calculate(5, 3, multiply));  // 15
```

Typed Arrays and Objects

Typed Arrays

Appending `[]` to a type name indicates an array of that type.

```
// Simple typed array
let numbers: number[] = [1, 2, 3, 4, 5];
// numbers.push("six");  // Error!

// Array of objects
interface Task {
    id: number;
    title: string;
    completed: boolean;
}

let tasks: Task[] = [
    { id: 1, title: "Learn TypeScript", completed: false },
    { id: 2, title: "Build an app", completed: false }
];
```

ReadonlyArray - can't be modified

The type argument in angle brackets (`<>`) specifies the `generic` element type. `ReadonlyArray` prevents changes through this reference.

```
let readOnlyScores: ReadonlyArray<number> = [95, 87, 92];
// readOnlyScores.push(88);  // Error!
// readOnlyScores[0] = 96;   // Error!
```

Typed Object Methods

```
interface StudentRecord {
    name: string;
    grades: number[];

    // Method signatures
    addGrade(grade: number): void;
    getAverage(): number;
}
```

```
class StudentImpl implements StudentRecord {
    name: string;
    grades: number[];

    constructor(name: string) {
        this.name = name;
        this.grades = [];
    }

    addGrade(grade: number): void {
        if (grade >= 0 && grade <= 100) {
            this.grades.push(grade);
        } else {
            throw new Error("Grade must be between 0 and 100");
        }
    }

    getAverage(): number {
        if (this.grades.length === 0) return 0;
        const sum = this.grades.reduce((a, b) => a + b, 0);
        return sum / this.grades.length;
    }
}
```

Generic Types

Generic function

```
function getFirstElement<T>(array: T[]): T | undefined {
    return array[0];
}

const firstNumber = getFirstElement([1, 2, 3]);
// returns 1
const firstName = getFirstElement(["Alice", "Bob"]);
// returns "Alice"
```

Generic class

```
class Stack<T> {
    private items: T[] = [];

    push(item: T): void {
        this.items.push(item);
    }

    pop(): T | undefined {
        return this.items.pop();
    }

    get size(): number {
        return this.items.length;
    }
}

const names = new Stack<string>();
names.push("Alice");
// names.push(42);  // Error! Argument of type 'number' is not assignable to 'string'
```

Control Flow with Types

Type Guards

typeof type guard

```
function processValue(value: string | number) {
    if (typeof value === "string") {
        // TypeScript knows value is string here
        console.log(value.toUpperCase());
    } else {
        // TypeScript knows value is number here
        console.log(value.toFixed(2));
    }
}
```

instanceof type guard

```
class Undergraduate {
    name: string;
    constructor(name: string) {
        this.name = name;
    }
}

class Professor {
    subject: string;
    constructor(subject: string) {
        this.subject = subject;
    }
}

function greetPerson(person: Undergraduate | Professor) {
    if (person instanceof Undergraduate) {
        console.log(`Hello student ${person.name}`);
    } else {
        console.log(`Hello professor of ${person.subject}`);
    }
}
```

Discriminated Unions

Common pattern for handling different cases. The `type` field tells TypeScript which case we have.

```
interface SuccessResult<T> {
    type: "success";
    data: T;
}

interface ErrorResult {
    type: "error";
    message: string;
}
```

```
type Result<T> = SuccessResult<T> | ErrorResult;

function handleResult(result: Result<number>) {
    switch (result.type) {
        case "success":
            // result is SuccessResult<number> here
            console.log("Data:", result.data.toFixed(1));
            break;
        case "error":
            // result is ErrorResult here
            console.error("Error:", result.message);
            break;
    }
}
```

We use a generic `T` instead of `any`, so `result.data` keeps its real type.

Null and Undefined Handling

strictNullChecks helps catch null errors

```
interface User {
    name: string;
    email?: string;  // Could be undefined
}

function sendEmail(user: User) {
    // TypeScript error: 'user.email' is possibly 'undefined'.
    // console.log(user.email.toLowerCase());

    // Correct way - check first
    if (user.email) {
        console.log(user.email.toLowerCase());
    }

    // Or use optional chaining
    console.log(user.email?.toLowerCase());
}
```

```
console.log(user.email?.toLowerCase());
```

If `user.email` is `undefined`, optional chaining skips the `toLowerCase()` call and returns `undefined`.

- Therefore, this line prints `undefined`.

Non-null assertion with `!` (use carefully!)

```
function processUser(user: User) {
    // Tell TypeScript you know email exists
    const email = user.email!;  // Use only when you're certain
    console.log(email.toLowerCase());
}
```

Because `email` is optional, its type includes `undefined`.

- The `!` tells TypeScript to treat it as defined so the code passes type-checking, but it does not change the value at runtime.

- Without `!`, TypeScript reports an error at `email.toLowerCase()` because `email` might be `undefined`. Check that it has a value before calling the method.
- If `email` is a string, `toLowerCase()` works; if it is `undefined`, the call throws a `TypeError`.

Recommended and safe

```
function processUser(user: User) {
    const email = user.email;
    if (email === undefined) {
        console.log("No email address available."); // Check availability
        return;
    }

    console.log(email.toLowerCase());
}
```

DOM Manipulation with Types

Typed DOM Selection

`document.getElementById` returns `HTMLElement | null`. Use `instanceof` to check the real element type **at runtime**; TypeScript then narrows the type for us.

```
const button = document.getElementById('submit-btn');

if (button instanceof HTMLButtonElement) {
    button.disabled = true;   // OK: button is an HTMLButtonElement here
}

// button.disabled = true;
// Error: 'button' is possibly 'null'.
```

Shortcut: Type Assertion (`as`)

`as` is an assertion, not a check: if the element is missing, it is `null` at runtime. Prefer the `instanceof` check on the previous slide.

```
const button = document.getElementById('submit-btn') as HTMLButtonElement;
const input = document.querySelector('#name-input') as HTMLInputElement;
const form = document.querySelector('form') as HTMLFormElement;
```

Specific DOM types let TypeScript provide relevant autocomplete.

- For example, `button` has a `disabled` property, `input` has a `value` property, and `form` has a `submit()` method.

```
button.disabled = true;
input.value = "TypeScript";
form.submit();
```

You can provide an expected element type to `querySelector`.

- It may not find a matching element, so its result can be `null`; check for `null` before using it.

```
const myDiv = document.querySelector<HTMLDivElement>("#my-div");
if (myDiv) {
    myDiv.style.backgroundColor = "blue";
}
```

Type-Safe Event Handling

Typed event listeners

```
const button = document.querySelector('#click-me') as HTMLButtonElement;

button.addEventListener('click', (event: MouseEvent) => {
    console.log(`Clicked at: ${event.clientX}, ${event.clientY}`);
});
```

TypeScript checks that `clientX` and `clientY` are valid `MouseEvent` properties, and that `disabled` is valid for an `HTMLButtonElement`.

This catches many mistakes while writing the code.

- However, `as HTMLButtonElement` is only a type assertion; it does not check the actual DOM element at runtime, so this example assumes `#click-me` is a button.

```
const target = event.currentTarget as HTMLButtonElement;
target.disabled = true;
```

```typescript
// Form events
const form = document.querySelector('#user-form') as HTMLFormElement;

form.addEventListener('submit', (event: Event) => {
    event.preventDefault();

    const formData = new FormData(form);
    const name = formData.get('name') as string;
    const age = parseInt(formData.get('age') as string);

    console.log({ name, age });
});

// Input events with proper types
const input = document.querySelector('#search') as HTMLInputElement;

input.addEventListener('input', (event: Event) => {
    const target = event.target as HTMLInputElement;
    const value: string = target.value;

    if (value.length < 3) {
        target.classList.add('error');
    } else {
        target.classList.remove('error');
    }
});
```

Creating Elements with Types

Type-safe element (in this example, HTMLLIElement) creation

```
function createTaskElement(task: Task): HTMLLIElement {
    const li = document.createElement('li');
    li.className = task.completed ? 'completed' : 'pending';

    const checkbox = document.createElement('input');
    checkbox.type = 'checkbox';
    checkbox.checked = task.completed;

    const label = document.createElement('label');
    label.textContent = task.title;

    li.appendChild(checkbox);
    li.appendChild(label);

    return li;
}
```

Using the function to create the HTMLLIElement.

```
const taskList = document.querySelector('#task-list') as HTMLULListElement;
const newTask: Task = { id: 1, title: "Learn TypeScript", completed: false };
const taskElement = createTaskElement(newTask);
taskList.appendChild(taskElement);
```

Advanced TypeScript Features

Type Assertions and Casting

Type assertion - telling TypeScript what type something is.

```
let someValue: any = "this is a string";
let strLength: number = (someValue as string).length;

// Alternative (old) syntax (not allowed in .tsx files)
let strLength2: number = (<string>someValue).length;

// Const assertions
const config = {
    endpoint: "https://api.example.com",
    timeout: 5000
} as const;  // Makes all properties readonly

// config.endpoint = "new";  // Error!
```

Literal Types

```
// String literal types
type Direction = "north" | "south" | "east" | "west";

function move(direction: Direction) {
    console.log(`Moving ${direction}`);
}

move("north");    // OK
// move("up");    // Error!

// Numeric literal types
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;

function rollDice(): DiceRoll {
    return Math.floor(Math.random() * 6 + 1) as DiceRoll;
}
```

Utility Types

Record - Object type with specific key/value types

An object whose keys are strings and whose values are numbers.

```
type Grades = Record<string, number>;
const courseGrades: Grades = {
    "Math": 85,
    "Science": 92,
    "English": 88
};
```

Exclude - remove types from union

```
type LetterGrade = "A" | "B" | "C" | "D" | "F";
type PassingGrade = Exclude<LetterGrade, "F">;  // "A" | "B" | "C" | "D"
```

Extract - keep only specified types

```
type HighGrade = Extract<LetterGrade, "A" | "B">;  // "A" | "B"
```

NonNullable - remove null and undefined

```
type MaybeString = string | null | undefined;
type DefiniteString = NonNullable<MaybeString>;  // string
```

Built-in utility types: Partial, Readonly, Pick, Omit

TypeScript already provides the mapped types we can write by hand (see the Appendix).

```
interface Profile {
    name: string;
    age: number;
    email: string;
}

type A = Partial<Profile>;                  // every property optional
type B = Readonly<Profile>;                 // every property readonly
type C = Pick<Profile, "name" | "email">;   // only these keys
type D = Omit<Profile, "email">;            // everything except email
```

Remember Sarah's journey:

- **JavaScript**: Fast to write, but runtime errors in production
- **TypeScript**: Slightly more code, but many mistakes are caught before running

TypeScript is not just about types—it's about building more reliable, maintainable software. As Sarah discovered, the initial learning investment pays off tremendously in reduced debugging time and increased confidence in your code!

The Grade Calculator example shows how TypeScript:

- Prevents type-related bugs
- Makes code more maintainable
- Provides confidence when refactoring
- Improves the development experience

Appendix

Important topics, but they are not needed for the Grade Calculator example.

- Function overloading
- Mapped types (how `Partial` and `Pick` work inside)
- Decorators

Overloading

```
// Function overloading - different parameter types
function processGrade(grade: number): string;
function processGrade(grade: string): number;
function processGrade(grade: number | string): string | number {
    if (typeof grade === "number") {
        // Convert number to letter grade
        if (grade >= 90) return "A";
        if (grade >= 80) return "B";
        if (grade >= 70) return "C";
        if (grade >= 60) return "D";
        return "F";
    }
    // Convert letter grade to its minimum number
    const minimum: Record<string, number> = { A: 90, B: 80, C: 70, D: 60, F: 0 };
    return minimum[grade] ?? 0;
}

console.log(processGrade(85));   // "B"
console.log(processGrade("A"));  // 90
```

Mapped Types

Usage 1: Make every property optional

```
{ name?: string; age?: number; email?: string; }
```

from existing interface:

```
interface Profile {
    name: string;
    age: number;
    email: string;
}
```

In this case, we can use Mapped Types

```
type MyPartial<T> = {
    [P in keyof T]?: T[P];
};
```

1. `keyof T`: gets all the property names of T.
2. `P in keyof T`: means "for each property P inside T".
3. `?:`: makes each property optional.
4. `T[P]`: means "use the same type as the original property".

```typescript
interface Profile {
    name: string;
    age: number;
    email: string;
}

type PartialProfile = MyPartial<Profile>;
// Same as: { name?: string; age?: number; email?: string; }

// Make all properties readonly
type MyReadonly<T> = {
    readonly [P in keyof T]: T[P];
};
```

Usage 2: We can create type with specific keys

For example, when we want to only select "name" and "email" from Profile interface, we can do

```
type MyPick<T, K extends keyof T> = {
    [P in K]: T[P];
};

type ProfileContact = MyPick<Profile, "name" | "email">;
// Same as: { name: string; email: string; }
```

Decorators

A **decorator** (a TC39 proposal that TypeScript supports) is a special function that can **wrap or modify**:

- It's like adding "extra behavior" to a method
- *without changing the method's original code*.

Example use:

- Logging, Validation, Timing, or Security checks

In this example, `LogMethod` adds logging around the `add` method.

```
class Calculator {
    @LogMethod
    add(a: number, b: number): number {
        return a + b;
    }
}
```

A method decorator receives the original method and a context object, then returns a replacement method.

```
function LogMethod(originalMethod: any, context: ClassMethodDecoratorContext) {
        const methodName = String(context.name);

        return function (this: any, ...args: any[]) {
                console.log(`Calling ${methodName} with args:`, args);
                const result = originalMethod.apply(this, args);
                console.log(`Result:`, result);
                return result;
        };
}
```

```
const calc = new Calculator();
calc.add(2, 3);

> Calling add with args: [2, 3]
> Result: 5
```

How it works?

1. @LogMethod is attached to add()
2. When add() is called:
    - Logs arguments
    - Calls original method
    - Logs result

